Design Elevator System

Functionality Requirements:
Object Identification:
Before moving ahead with UML, lets first understand:
Elevator Selection Strategy:
Elevator car moving behavior (i.e requests should be ordered by direction):
SCAN Algorithm:
LOOK Algorithm:
UML:
Implementation

Resources

- 8. Elevator System, Low Level Design (Hindi) | SDE LLD interview question | Design Elevator System
- Code Repo → src/main/java/com/conceptcoding/interviewquestions/elevator · main · shrayansh jain / LLD-LowLevelDesign · GitLab

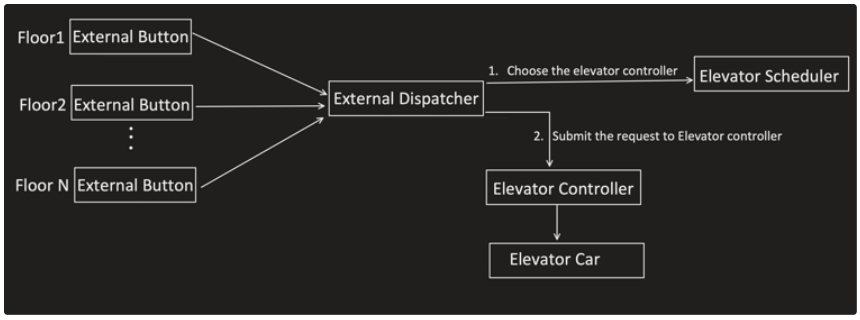
Functionality Requirements:

- A building has multiple **elevators** and multiple **floors**.
- A user can request an elevator **externally** using **Up / Down** buttons at each floor.
- These Up/ Down direction button is used to **choose the best Elevator** to server the request.
- One Elevator is chosen, the floor is added in that particular elevator **bucket list**.
- A user inside an elevator can also press an **internal button** to select destination floor.
- Request generated from elevator Internal button, should always server by the same elevator only.
- Elevators should **remain idle (sleep) when no requests exist**, and **wake** when **new request arrives**.
- Requests should be **ordered by direction**:
  - Going up → visit floors in ascending order
  - Going down → visit floors in descending order

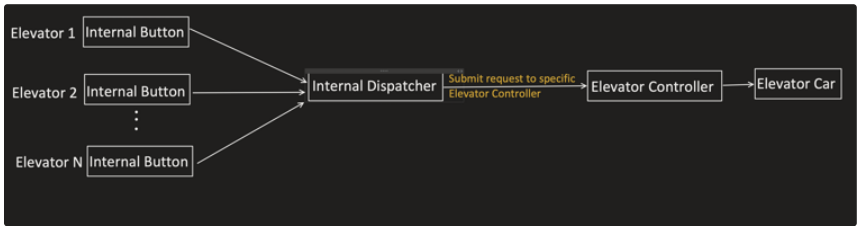
Concept && Coding

Object Identification:

- Building
- Floor
- Elevator - just a POJO
- Elevator Controller - each elevator is controlled by its controller, which manages its requests.
- External Button - Each floor has the External Button
- Internal Button - Each Elevator car has 1 Internal Button
- Elevator Scheduler - Maintains the List of Elevator Controllers + also has Elevator Selection Strategy to choose the best Elevator to serve the request
- External Dispatcher → bridge between External Buttons and Elevator Controller



- Internal Dispatcher → Internal buttons can directly call their Elevator Controller, but sometimes we need to put logging and validation, so its better to have one proxy. Also its follows the same path as external, so its clear too.



Before moving ahead with UML, lets first understand:

1. Elevator Selection Strategy?
   and
2. Elevator moving behavior (i.e requests should be **ordered by direction)**, what does that mean?
   - Going up → visit floors in ascending order
   - Going down → visit floors in descending order

Concept && Coding

Elevator Selection Strategy:

- When External Button is pressed, info which we have is:
  - **Floor**: floor at which user is waiting for the lift
  - **Direction**: whether user wants to go up or down.

Now when there are more than 1 elevator, then we need to select the Elevator based on some strategy like:

- Nearest Elevator : the elevator which is nearest to the user, should handle the request.
- Least Busy Elevator: the elevator which has least number of request, should handle the request

etc.

So, Selection strategy uses :

- Floor info, from where the request has come.
- Direction info, where the user wants to go
- Elevators info like:
  - its request bucket
  - Elevator direction
  - Elevator current floor

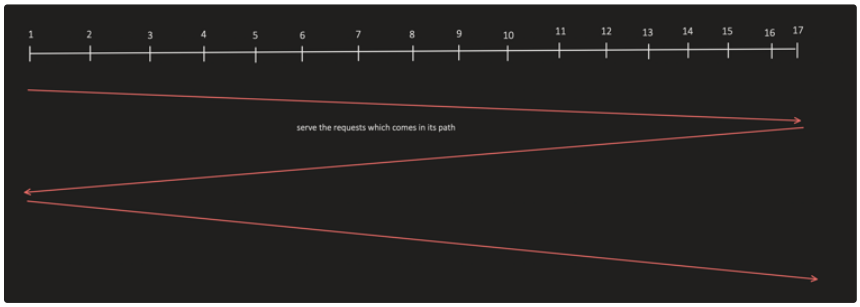
And based on the algorithm, it picks one elevator to serve the request.

Elevator car moving behavior (i.e requests should be ordered by direction):

SCAN Algorithm:

- Head moves in one direction first, servicing the requests along the way.
- Once it reached till the end, it reverses the direction. So it moves End to end.
  EX:

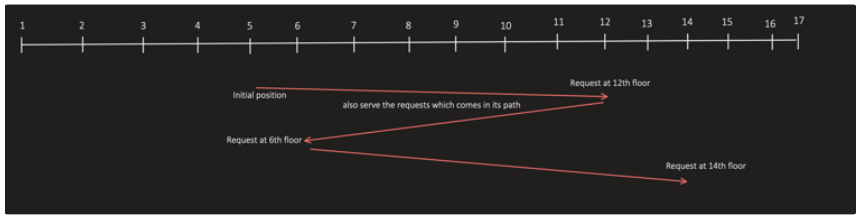
  Elevator going till top floor even if there is no request.



LOOK Algorithm:

Same as SCAN, but with one change that, it Don't go till end, stop when there is no request.

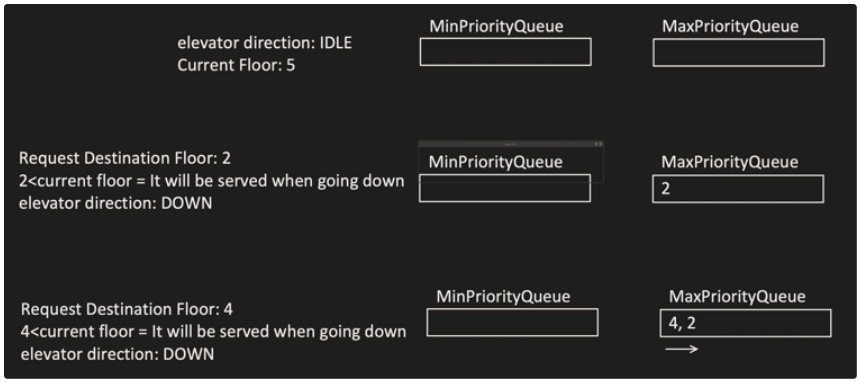
Concept && Coding

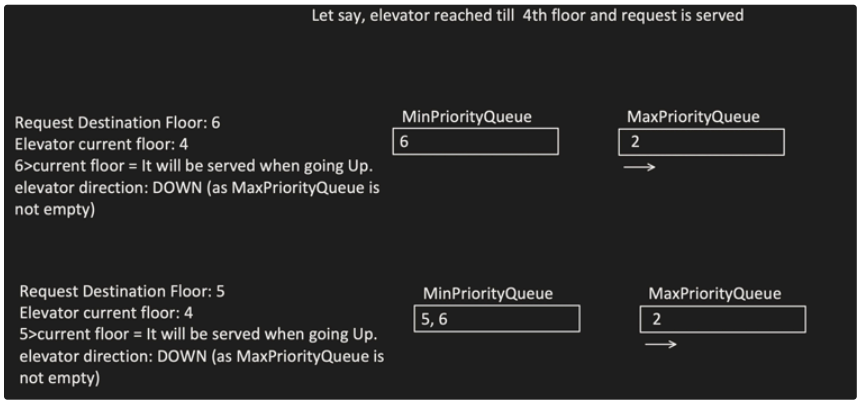


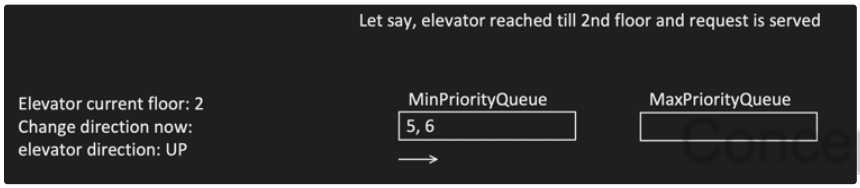
We can implement this LOOK Algorithm using 2 priority queues:

- Min Priority Queue: Used when elevator is going UP, as it need to serve the request in ascending order.
- Max Priority Queue: Used when elevator is going DOWN, as it need to serve the request in descending order.

Example:



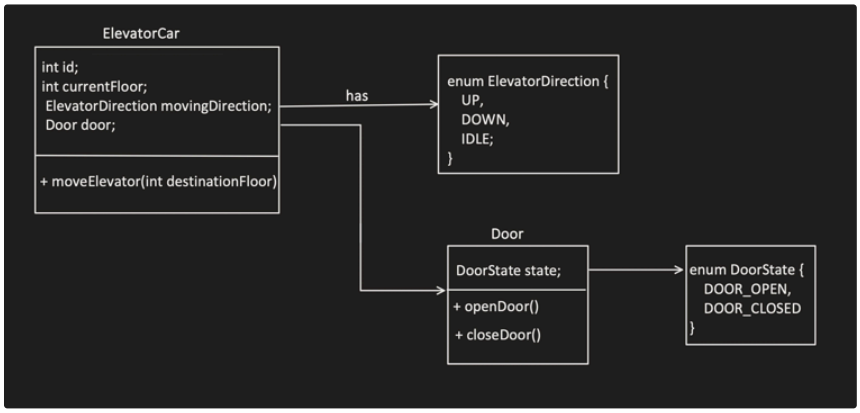




Concept && Coding

UML:

1. **Elevator Car:** An object, which moves to particular destination when asked to.




```java
package com.conceptcoding.interviewquestions.elevator;

import com.conceptcoding.interviewquestions.elevator.enums.ElevatorDirection;

public class ElevatorCar {

    int id;
    int currentFloor;
```

```java
    int nextFloorStoppage;
    ElevatorDirection movingDirection;
    Door door;

    public ElevatorCar(int id) {
        this.id = id;
        currentFloor = 0;
        movingDirection = ElevatorDirection.IDLE;
        door = new Door();
    }

    public void showDisplay() {
        System.out.println("elevator:" + id + " Current floor: " + currentFloor + " going: " + movingDirection);
    }

    public void moveElevator(int destinationFloor) {
        //this is dump object, so if command has come, to go particular direction and particular floor, it just move
        //no matter what its current state and floor.

        this.nextFloorStoppage = destinationFloor;
        if (this.currentFloor == nextFloorStoppage) {
            door.openDoor(id);
            return;
        }

        int startFloor = this.currentFloor;
        door.closeDoor(id);
        if(nextFloorStoppage >=currentFloor) {
            movingDirection = ElevatorDirection.UP;
            showDisplay();
            //+1 i am doing bcoz, floor start from 0,1,2.... so if anyone goes from 1st floor to 2nd, so only 1 floor
            //lift has to move, not 2.
            for (int i = startFloor+1; i<= nextFloorStoppage; i++) {
                try {
                    Thread.sleep(5);
                }catch (Exception e) {

                }
                setCurrentFloor(i);
                showDisplay();
            }
        }
        else  {
            movingDirection = ElevatorDirection.DOWN;

            showDisplay();
            for (int i = startFloor-1; i>= nextFloorStoppage; i--) {
                try {
                    Thread.sleep(5);
                }catch (Exception e) {

                }
                setCurrentFloor(i);
                showDisplay();
            }
        }
        door.openDoor(id);
    }

    public void setCurrentFloor(int currentFloor) {
        this.currentFloor = currentFloor;
    }
}


```

Concept && Coding

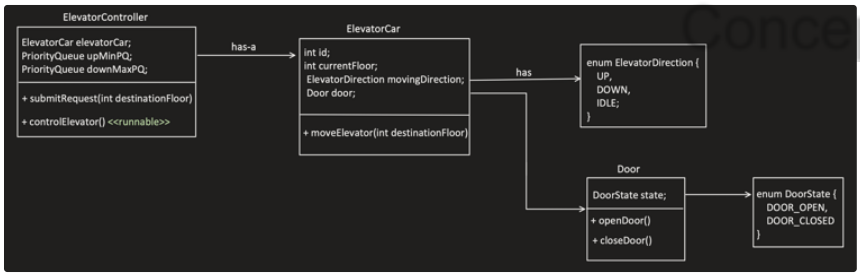
```java
public enum ElevatorDirection {
    UP,
```

```
    DOWN,
    IDLE;
}
```

```
public class Door {
    private DoorState doorState;

    Door() {
        doorState = DoorState.DOOR_CLOSED;
    }

    public void openDoor(int id) {
        doorState = DoorState.DOOR_OPEN;
        System.out.println("Opening the Elevator door of elevator:" + id);
    }

    public void closeDoor(int id) {
        doorState = DoorState.DOOR_CLOSED;
        System.out.println("Closing the Elevator door of elevator:" + id);
    }

}
```

```
public enum DoorState {
    DOOR_OPEN,
    DOOR_CLOSED
}
```

2. **Elevator Controller**



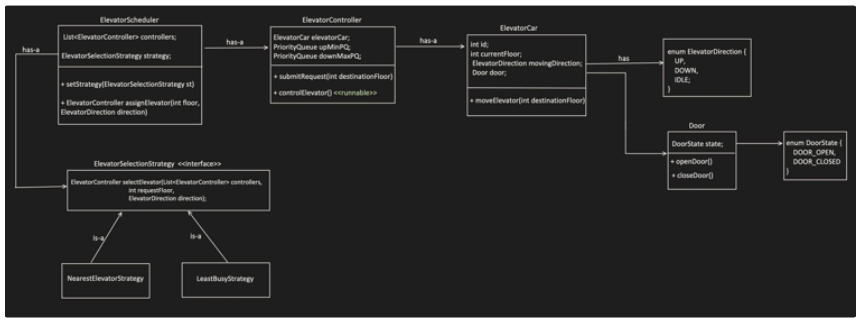
Concept && Coding

```
package com.conceptcoding.interviewquestions.elevator;

import com.conceptcoding.interviewquestions.elevator.enums.ElevatorDirection;

import java.util.concurrent.PriorityBlockingQueue;

public class ElevatorController implements Runnable {

    PriorityBlockingQueue<Integer> upMinPQ;
    PriorityBlockingQueue<Integer> downMaxPQ;

    ElevatorCar elevatorCar;

    private final Object monitor = new Object();

    ElevatorController(ElevatorCar elevatorCar) {

        this.elevatorCar = elevatorCar;
        upMinPQ = new PriorityBlockingQueue<>();
        downMaxPQ = new PriorityBlockingQueue<>(10, (a, b) -> b - a);
    }

    public void submitRequest(int destinationFloor) {
        enqueueRequest(destinationFloor);
```

```java
    }

    private void enqueueRequest(int destinationFloor) {
        System.out.println("Request details-> destinationFloor: " + destinationFloor + " accepted by elevator:" + elevatorCar.id);

        if (destinationFloor == elevatorCar.nextFloorStoppage){
            return;
        }
        if (destinationFloor >= elevatorCar.nextFloorStoppage) {
            if (!upMinPQ.contains(destinationFloor)) {
                upMinPQ.offer(destinationFloor);
            }
        } else {
            if (!downMaxPQ.contains(destinationFloor)) {
                downMaxPQ.offer(destinationFloor);
            }
        }

        synchronized (monitor) {
            monitor.notify();   // wake elevator thread
        }
    }

    @Override
    public void run() {
        controlElevator();
    }

    public void controlElevator() {

        while (true) {

            //no request, go to sleep
            synchronized (monitor) {
                while (upMinPQ.isEmpty() && downMaxPQ.isEmpty()) {
                    try {
                        System.out.println("elevator:" + elevatorCar.id + " is IDLE");
                        elevatorCar.movingDirection = ElevatorDirection.IDLE;
                        monitor.wait(); // sleep until request arrives
                    } catch (InterruptedException e) {
                        Thread.currentThread().interrupt();
                    }
                }
            }


            while (!upMinPQ.isEmpty()) {
                int floor = upMinPQ.poll();
                System.out.println("Serving floor: " + floor + " by elevator:" + elevatorCar.id + " currentFloor: " + elevatorCar.currentFloor);
                elevatorCar.moveElevator(floor);
            }


            while (!downMaxPQ.isEmpty()) {
                int floor = downMaxPQ.poll();
                System.out.println("Serving floor: " + floor + " by elevator:" + elevatorCar.id + " currentFloor: " + elevatorCar.currentFloor);
                elevatorCar.moveElevator(floor);
            }
        }
    }
}
```

Concept && Coding

3. **Elevator Scheduler:**




```java
public class ElevatorScheduler {

    private final List<ElevatorController> controllers;
    private ElevatorSelectionStrategy strategy;

    public ElevatorScheduler(List<ElevatorController> controllers,
                             ElevatorSelectionStrategy strategy) {
        this.controllers = controllers;
        this.strategy = strategy;
    }

    public void setStrategy(ElevatorSelectionStrategy strategy) {
        this.strategy = strategy;
    }

    public ElevatorController assignElevator(int floor, ElevatorDirection direction) {
        return strategy.selectElevator(controllers, floor, direction);
    }
}
```

Concept && Coding

```java
public interface ElevatorSelectionStrategy {

    ElevatorController selectElevator(List<ElevatorController> controllers,
                                      int requestFloor,
                                      ElevatorDirection direction);
}
```

```java
package com.conceptcoding.interviewquestions.elevator;

import com.conceptcoding.interviewquestions.elevator.enums.ElevatorDirection;

import java.util.List;

public class NearestElevatorStrategy implements ElevatorSelectionStrategy {

    @Override
    public ElevatorController selectElevator(List<ElevatorController> controllers,
                                             int requestFloor,
                                             ElevatorDirection direction) {

        ElevatorController best = null;
        int minDistance = Integer.MAX_VALUE;

        //1. Pick the one which is going in same direction and minimum distance from the destination
```

```java
        for (ElevatorController controller : controllers) {
            int nextFloorStoppage = controller.elevatorCar.nextFloorStoppage;

            // Good candidate if moving same direction & not passed requested floor
            boolean isSameDirectionCandidate =
                    controller.elevatorCar.movingDirection == direction &&
                            ((direction == ElevatorDirection.UP && nextFloorStoppage <= requestFloor) ||
                                    (direction == ElevatorDirection.DOWN && nextFloorStoppage >= requestFloor));

            int dist = Math.abs(nextFloorStoppage - requestFloor);

            if (isSameDirectionCandidate && dist < minDistance) {
                minDistance = dist;
                best = controller;
            }
        }

        // fallback: if not able to choose, pick the idle one
        if (best == null) {
            for (ElevatorController controller : controllers) {
                if(controller.elevatorCar.movingDirection == ElevatorDirection.IDLE) {
                    best = controller;
                    break;
                }
            }

            //reached here means, no list is going in same direction and no lift is IDLE too, then pick any lift
            if(best == null) {
                best = controllers.get(0);
            }
        }
        return best;
    }
}

```

Concept && Coding

```java
package com.conceptcoding.interviewquestions.elevator;

import com.conceptcoding.interviewquestions.elevator.enums.ElevatorDirection;

import java.util.List;

public class LeastBusyStrategy implements ElevatorSelectionStrategy {

    @Override
    public ElevatorController selectElevator(List<ElevatorController> controllers,
                                             int requestFloor,
                                             ElevatorDirection direction) {

        ElevatorController best = null;
        int minLoad = Integer.MAX_VALUE;

        for (ElevatorController controller : controllers) {
            int load = controller.upMinPQ.size() +
                    controller.downMaxPQ.size();

            if (load < minLoad) {
                minLoad = load;
                best = controller;
            }
        }
```

```
        return best;
    }
}

```

4. **Button and dispatchers:**

```
package com.conceptcoding.interviewquestions.elevator;

import com.conceptcoding.interviewquestions.elevator.enums.ElevatorDirection;

public class ExternalButton {

    private final ExternalDispatcher dispatcher;

    public ExternalButton(ExternalDispatcher dispatcher) {
        this.dispatcher = dispatcher;
    }

    // this direction of external button is only helpful in selecting the correct elevator
    public void pressButton(int floor, ElevatorDirection direction) {
        dispatcher.submitExternalRequest(floor, direction);
    }
}


```

Concept && Coding

```
package com.conceptcoding.interviewquestions.elevator;

import com.conceptcoding.interviewquestions.elevator.enums.ElevatorDirection;

import java.util.List;

public class ExternalDispatcher {

    ElevatorScheduler scheduler;

    public ExternalDispatcher(ElevatorScheduler scheduler) {
        this.scheduler = scheduler;
    }

    public void submitExternalRequest(int floor, ElevatorDirection direction) {

        ElevatorController controller =
                scheduler.assignElevator(floor, direction);
```

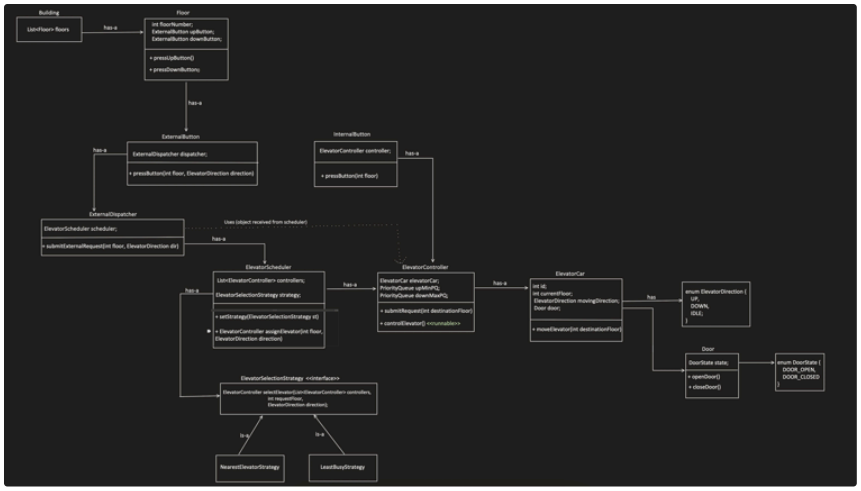
```
        controller.submitRequest(floor);
    }

}

```

```java
package com.conceptcoding.interviewquestions.elevator;


public class InternalButton {

    private final ElevatorController controller;

    public InternalButton(ElevatorController controller) {
        this.controller = controller;
    }

    public void pressButton(int destinationFloor) {
        //we can also remove teh Internal dispatcher from mid, but generally say for validation, controller and
        //similar code flow like external button, its good have

        InternalDispatcher.getInstance()
                .submitInternalRequest(destinationFloor, controller);
    }
}
```

```java
package com.conceptcoding.interviewquestions.elevator;

public class InternalDispatcher {

    private static InternalDispatcher INSTANCE = new InternalDispatcher();

    private InternalDispatcher() {}

    public static InternalDispatcher getInstance() {
        return INSTANCE;
    }

    // elevatorController is known based on button press origin
    public void submitInternalRequest(int destinationFloor, ElevatorController controller) {
        controller.submitRequest(destinationFloor);
    }
}
```

Concept && Coding

4. **Floors and Building:**




```java
package com.conceptcoding.interviewquestions.elevator;

import
com.conceptcoding.interviewquestions.elevator.enums.ElevatorDirection;

public class Floor {

    int floorNumber;
    ExternalButton upButton;
    ExternalButton downButton;

    public Floor(int floorNumber, ExternalDispatcher dispatcher) {
        this.floorNumber = floorNumber;
        this.upButton = new ExternalButton(dispatcher);
        this.downButton = new ExternalButton(dispatcher);
    }

    public void pressUpButton() {
        upButton.pressButton(floorNumber, ElevatorDirection.UP);
    }

    public void pressDownButton() {
        downButton.pressButton(floorNumber, ElevatorDirection.DOWN);
    }
}
```

Concept && Coding

```java
package com.conceptcoding.interviewquestions.elevator;

import java.util.ArrayList;
import java.util.List;

public class Building {

    List<Floor> floors = new ArrayList<>();

    public Building(int totalFloors, ExternalDispatcher dispatcher) {
        for (int i = 1; i <= totalFloors; i++) {
            floors.add(new Floor(i, dispatcher));
        }
    }

    public Floor getFloor(int floor) {
        return floors.get(floor-1);
    }
}
```

Implementation

Refer Code Repository for executable code → src/main/java/com/conceptcoding/interviewquestions/elevator · main · shrayansh jain / LLD-LowLevelDesign · GitLab

Concept && Coding